



News in Mail

Pick your interests. Get a short, AI-summarized news briefing by email every morning and evening.

Project documentation

Live demo: <https://news-in-mail-production.up.railway.app>

Demo login: **demo@newsinmail.ng** / **demo1234**

1. Overview
2. How the core logic works
3. Architecture and data model
4. Screen walkthrough
5. Mobile view
6. Running locally
7. Testing
8. Deployment
9. 5-minute presentation script
10. Decisions and limits

1. Overview

News in Mail is a web app where readers choose the subjects they care about (Nigeria, World, Business, Technology, Sports and so on) and when they want to read. Twice a day the app collects stories from free RSS news feeds, picks the most recent ones for each reader, asks an AI model to summarize each story in one or two sentences, and sends a clean email briefing.

The site itself looks and works like a news reader: a home page with top stories, a tab for each topic, search, an inbox of every email sent, and a settings page.

Main features

- Email and password accounts, with the demo account filled in on the sign-in page.
- Onboarding: pick topics, morning and evening times and time zone, then a test email is sent straight away.
- Live stories from 20 free feeds: BBC, The Guardian, Al Jazeera, TechCrunch, Premium Times, Punch, Channels TV and Vanguard.
- AI summaries through OpenRouter (Claude Sonnet 5.5, with Gemini Flash as fallback). Each story is summarized once and cached.
- Scheduler that sends the morning and evening briefings in each reader's own time zone, without duplicates.
- Inbox that records every briefing as sent, failed or skipped, and shows the exact email.
- Works on phones: single-column layout, swipeable tabs, no sideways scrolling.

Users in the demo data

Name	Email	State
Chiamaka Okafor	demo@newsinmail.ng / demo1234	Active. Follows Nigeria, Technology, Business, Sports. Five days of email history including one failed and one skipped briefing.
Tunde Bakare	tunde.bakare@example.ng / tunde1234	Briefings paused, so the scheduler skips him.
Aisha Bello	aisha.bello@example.ng / aisha1234	Signed up but has not finished onboarding, so she gets nothing yet.

2. How the core logic works

All business rules are pure functions in `src/lib/`. They take the current time as an argument, so tests and demos can pin the clock with `NEWSINMAIL_TODAY=YYYY-MM-DD` (or `NEWSINMAIL_NOW` for an exact instant).

Step 1: collect stories (`src/server/ingest.ts`)

Every 5 minutes the scheduler checks whether the feeds were read in the last 20 minutes. If not, it downloads all 20 RSS feeds in parallel, cleans the text (HTML, typographic quotes and dashes, "Read More" links), and stores up to 15 items per feed. The story link is unique, so the same story is never stored twice. Stories older than 14 days are deleted.

Step 2: decide who is due (`src/lib/schedule.ts`, `dueSlots`)

For each reader who has finished onboarding and is not paused, the app converts "now" into the reader's local date and time. A slot (morning or evening) is due when:

- the slot is switched on,
- its local time has passed today,
- it is less than 3 hours late (so a server restart at night does not send a stale morning email), and
- nothing has been recorded for that slot today.

The database backs this up with a unique index on (user, local date, slot), so even two servers running at once cannot send the same briefing twice.

Step 3: pick stories (`src/lib/ranking.ts`, `selectArticles`)

- Keep only the reader's topics and stories from the last 36 hours.
- Remove any story the reader already received in the last 3 days.
- Sort newest first, then take stories round-robin across topics (at most 3 per topic, 10 in total) so one busy topic cannot fill the email.

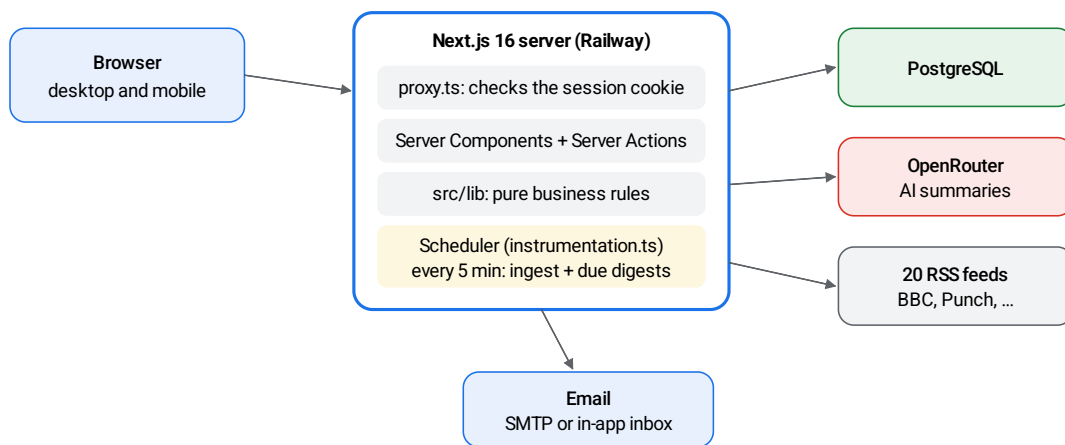
Step 4: summarize (`src/server/openrouter.ts`)

Stories that do not yet have a summary are sent to OpenRouter in one request. The model returns JSON with a one-line intro and a summary for each story. Summaries are saved on the story so they are reused by every reader. If the AI call fails or there is no API key, the app falls back to the first one or two sentences of the feed excerpt, so an email always goes out.

Step 5: render and deliver (`src/lib/email.ts`, `src/server/mailer.ts`)

The email is built with tables and inline styles, which mail apps need. It greets the reader by first name, groups stories by topic in the reader's order, links each headline to the source and says when the next briefing is due. If `SMTP_URL` is set the email is sent by SMTP. Otherwise it is stored in the in-app inbox. The result is recorded as **sent**, **failed** (with the mail server error) or **skipped** (no new stories).

3. Architecture and data model



Tech stack

Part	Choice
Web framework	Next.js 16 (App Router, Server Components, Server Actions), React 19, TypeScript strict
Database	PostgreSQL with Drizzle ORM. drizzle-kit creates versioned SQL migrations in <code>drizzle/</code>
Validation	Zod schemas in <code>src/lib/validation.ts</code> , with errors shown under each field
Auth	bcryptjs password hashes. A signed JWT (jose, HS256, 7 days) in an HTTP-only cookie. <code>src/proxy.ts</code> guards private pages
UI	Plain CSS, lucide-react icons, DiceBear avatars generated locally, Roboto from @fontsource, custom SVG logo and illustrations
Tests	Vitest (unit and integration against real Postgres) and Playwright (desktop and mobile)

Data model

Table	Key columns	Purpose
users	id, name, email (unique), password_hash, timezone, morning_enabled, morning_time, evening_enabled, evening_time, paused, onboarded_at	Accounts and delivery preferences
interests	user_id, topic (primary key on both)	Which topics each user follows
articles	id, link (unique), topic, source, title, excerpt, summary, summary_model, is_sample, published_at	Stories from feeds, with cached AI summary
digests	id, user_id, slot, digest_date, status, subject, html, text, article_ids, delivered_via, error	Every briefing produced. Unique on (user_id, digest_date, slot) for morning and evening

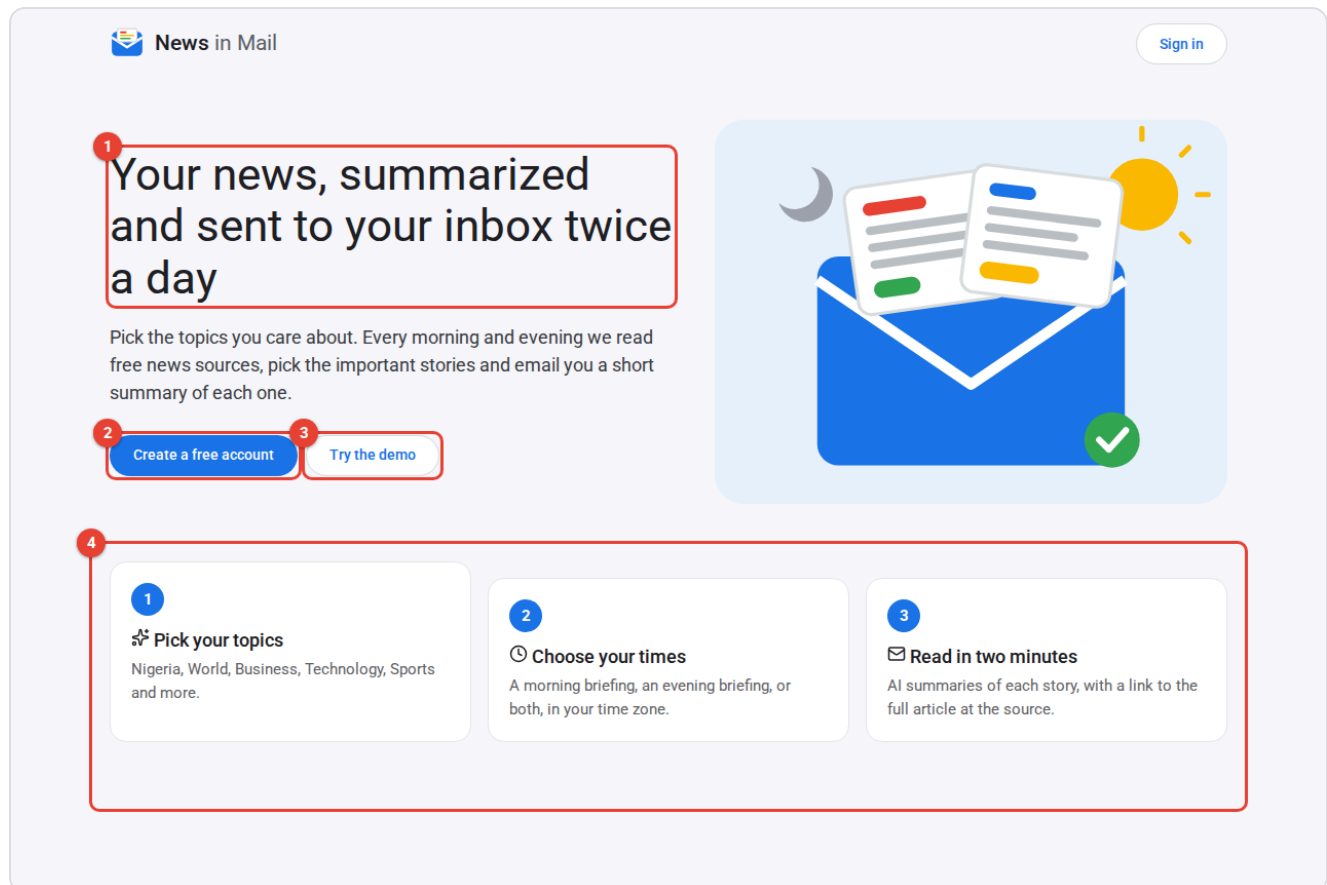
Every query that reads digests filters by the signed-in user's id, and opening another user's email returns "not found".

4. Screen walkthrough

The numbers on each screenshot match the list under it. Screenshots use the seeded demo data with the clock fixed at 3:00 PM Lagos time.

1. Landing page

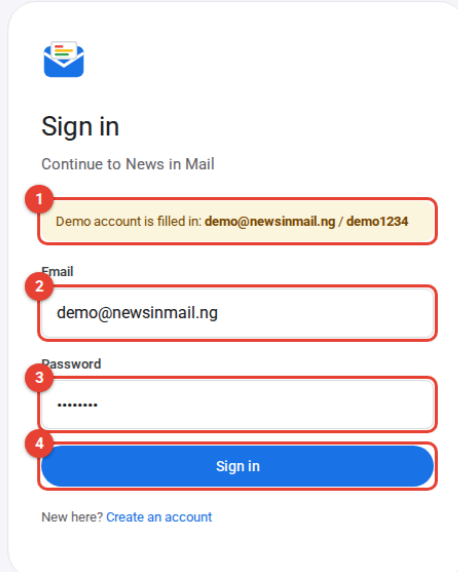
The first page a visitor sees. It explains the product in one sentence and offers two ways in.



1. Headline that says what the app does.
2. Starts sign up, then onboarding.
3. Goes to sign in with the demo account filled in.
4. The three steps: pick topics, choose times, read.

2. Sign in

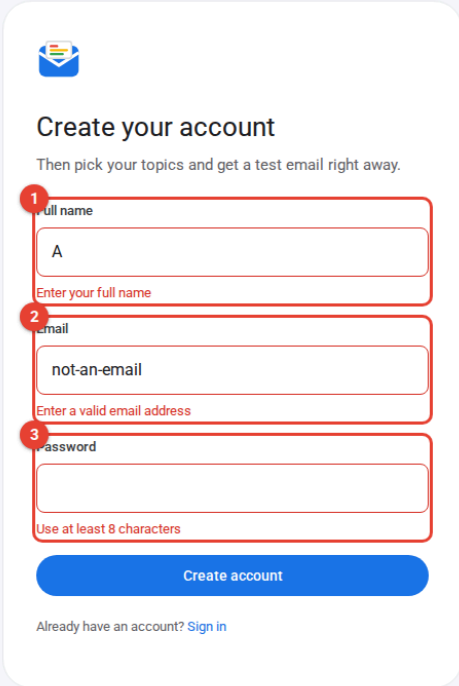
Email and password sign in. For the presentation the demo account is filled in already.

A screenshot of a 'Sign in' form for 'News in Mail'. The form is white with rounded corners and is centered on a light gray background. At the top left is a logo consisting of a blue envelope with a yellow and red flag. Below the logo, the text 'Sign in' is in bold, followed by 'Continue to News in Mail' in a smaller font. There are four numbered red circles indicating steps: 1. A yellow box contains the text 'Demo account is filled in: demo@newsinmail.ng / demo1234'. 2. The 'Email' field is pre-filled with 'demo@newsinmail.ng'. 3. The 'Password' field is pre-filled with '*****'. 4. The 'Sign in' button is highlighted. Below the button is a link that says 'New here? Create an account'.

- 1 Demo credentials are shown on the page.
- 2 Pre-filled demo email.
- 3 Pre-filled demo password.
- 4 Signs in and sets a signed JWT in an HTTP-only cookie.

3. Sign up with validation errors

Forms are validated on the server with Zod. Errors appear under each field and the typed values are kept.



Create your account

Then pick your topics and get a test email right away.

1 **Full name**
A
Enter your full name

2 **Email**
not-an-email
Enter a valid email address

3 **Password**
Use at least 8 characters

Create account

Already have an account? [Sign in](#)

- 1 Name too short.
- 2 Invalid email. The value stays in the box.
- 3 Password rule shown inline.

4. Onboarding: pick topics and times

New users land here right after sign up. Nothing is sent until they finish this form.

News in Mail

Welcome, Adaeze

Pick your topics and delivery times. When you finish we will send you a test email straight away so you can see what your briefings look like.

Topics
Pick the subjects you want in your briefings.

1. Topic chips: World, Nigeria, Africa, Business, Technology, Science, Health, Sports, Entertainment. Selected topics (World, Nigeria, Technology) are highlighted in blue.

Delivery times
We send at most two emails a day.

2. Morning briefing switch and time (7:00 AM)

3. Evening briefing switch and time (6:00 PM)

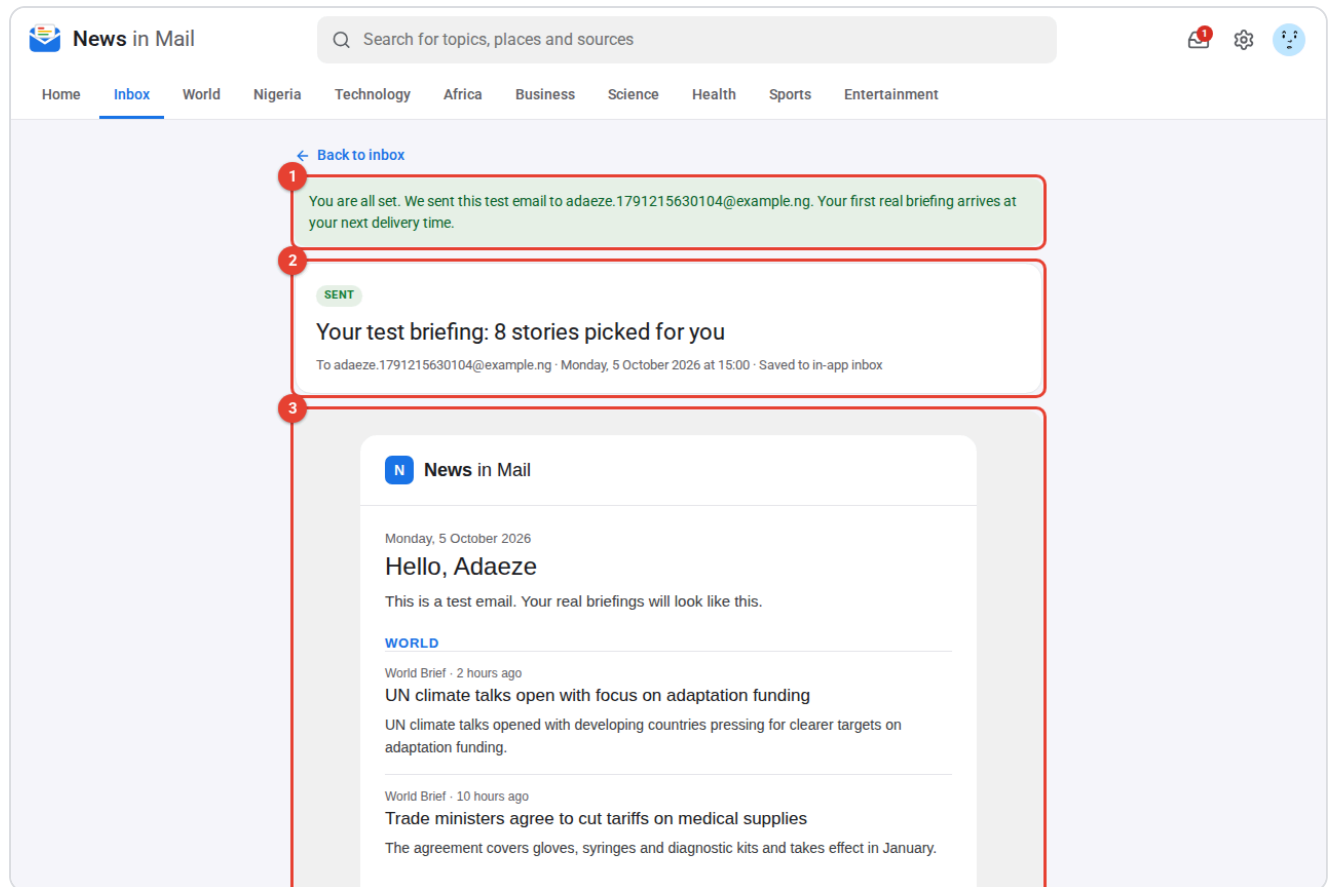
4. Time zone: Africa/Lagos

5. Finish and send test email button

- 1 Topic chips. Selected topics turn blue.
- 2 Morning briefing switch and time (before noon).
- 3 Evening briefing switch and time (noon or later).
- 4 Time zone. Africa/Lagos by default.
- 5 Saves preferences and sends the test email.

5. The test email

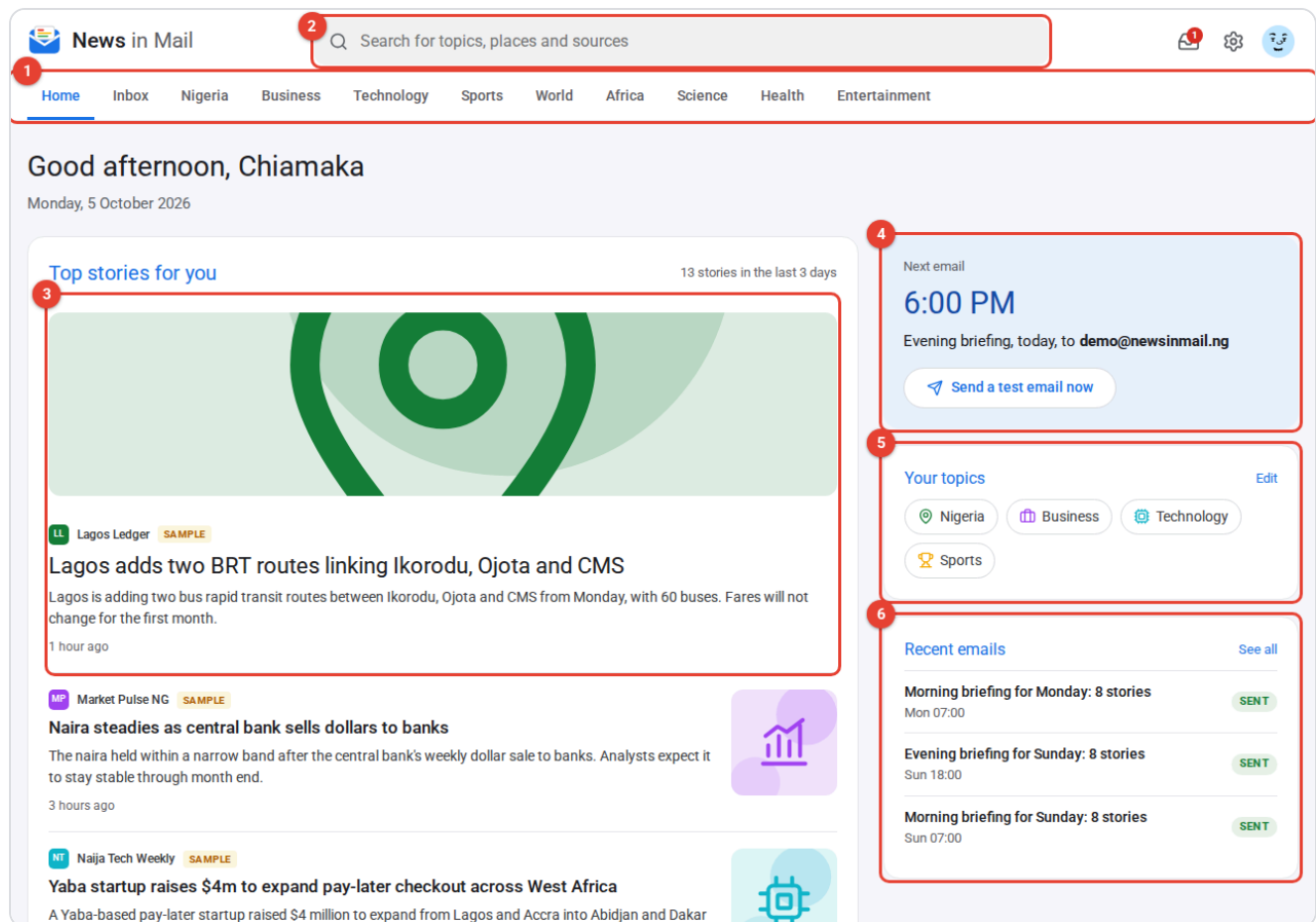
As soon as onboarding is done the user sees the email we just sent, exactly as it looks in a mail app.



- 1 Confirmation with the address it went to.
- 2 Subject, recipient, time and delivery method.
- 3 The rendered email: greeting, AI intro and stories grouped by topic.

6. Home: your briefing

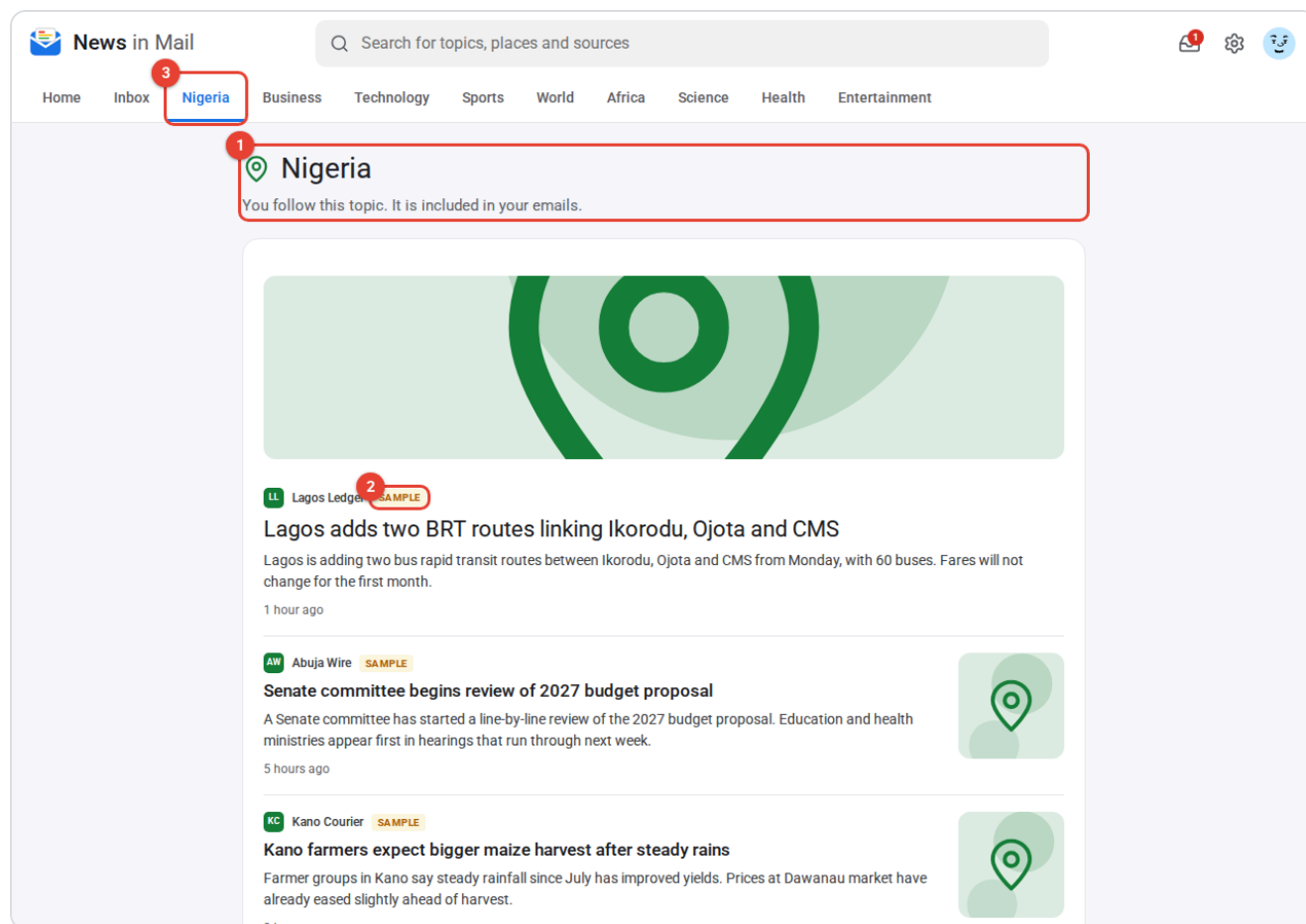
A Google News style home page with the stories that will go into the next emails.



- 1 Section tabs: Home, Inbox and every topic. Followed topics come first.
- 2 Search across all stored stories.
- 3 Lead story with an illustration, source, summary and age.
- 4 When the next email goes out, plus a button to send a test now.
- 5 The reader's topics. Edit goes to Settings.
- 6 The last three emails with their status.

7. Topic page

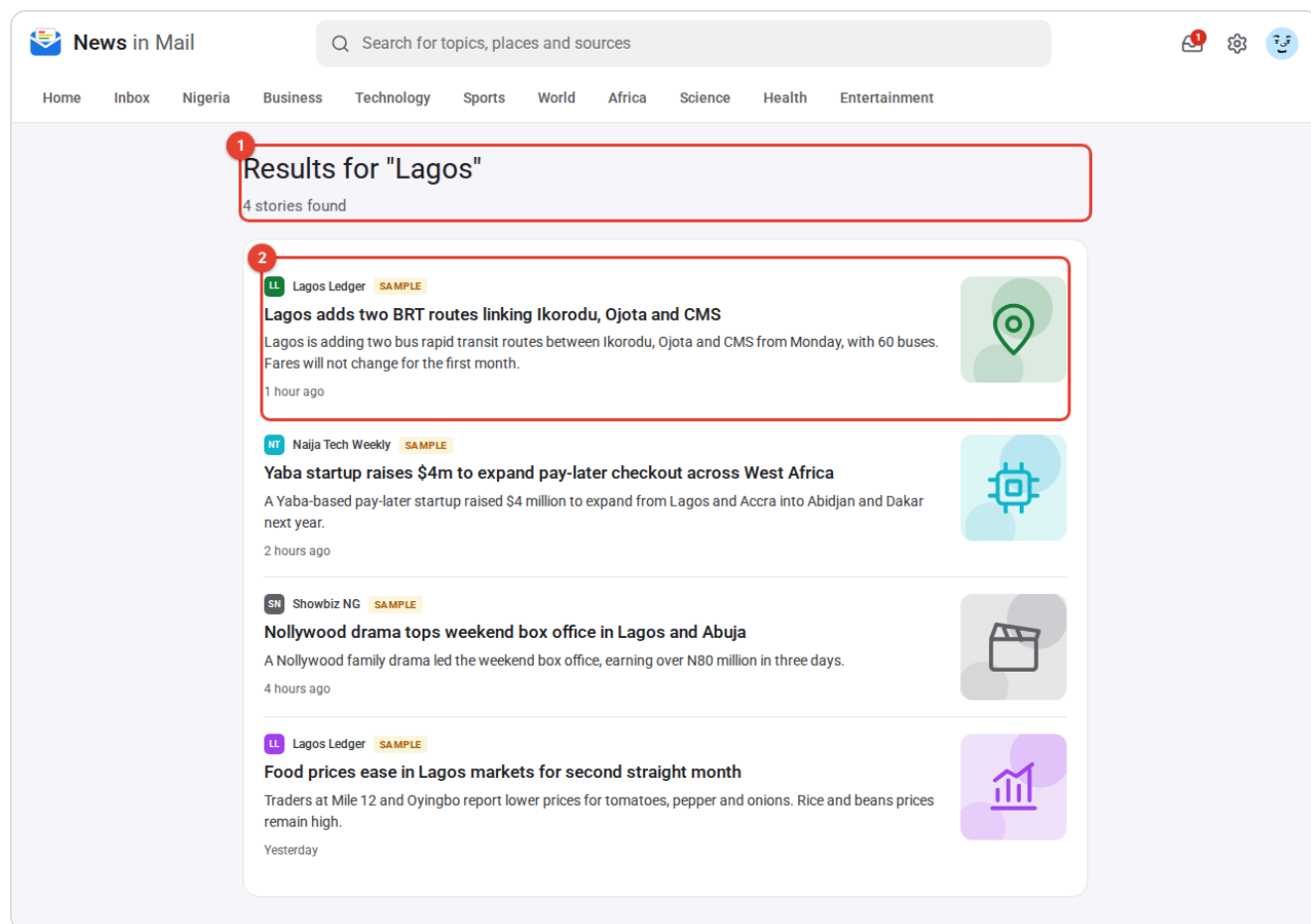
Every topic has its own page with the latest stories from its sources.



- 1 Topic name and whether it is in your emails.
- 2 Sample badge: seeded stories are marked so they are never mistaken for real news.
- 3 Active tab is underlined in blue.

8. Search

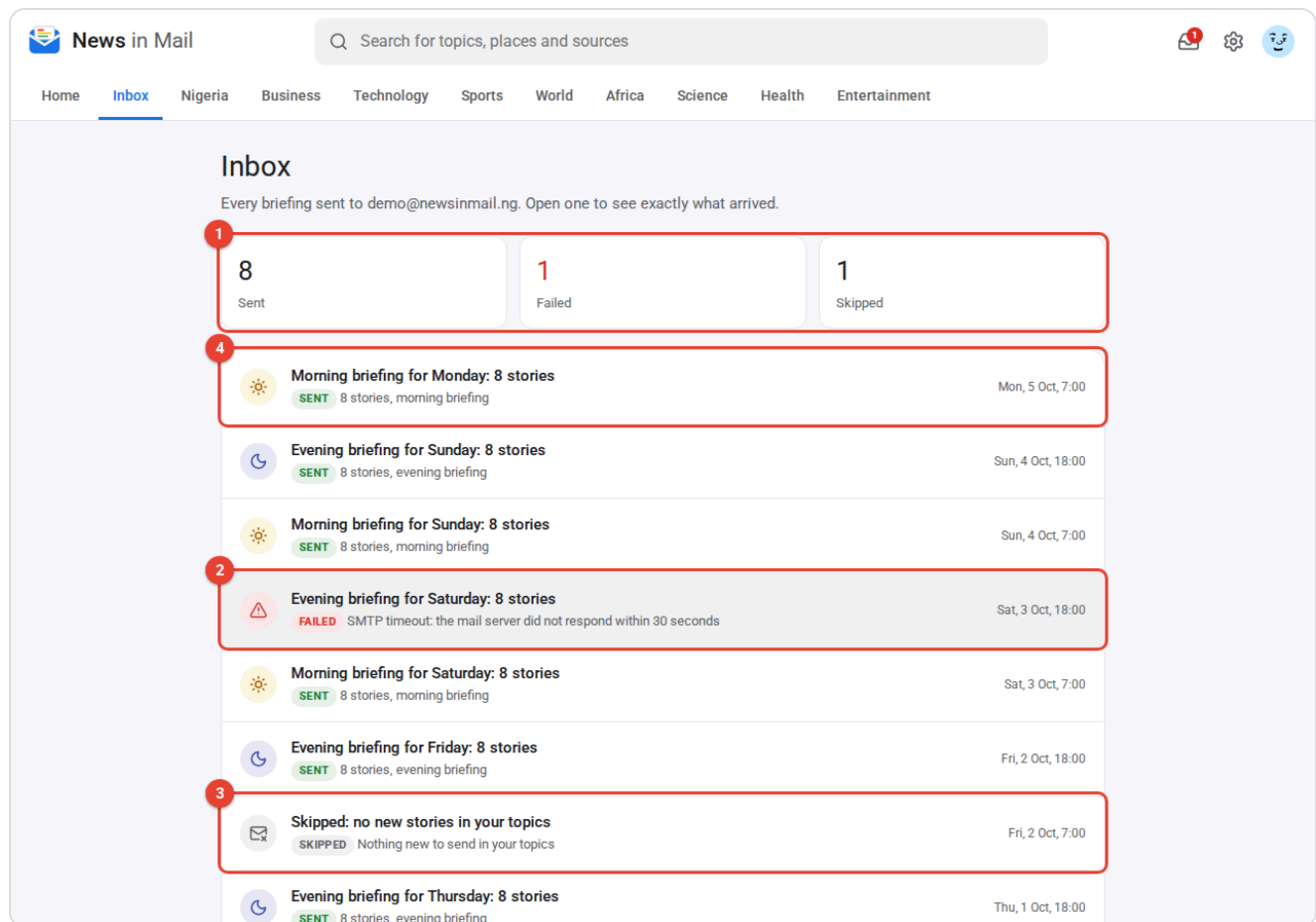
A simple case-insensitive search over titles, excerpts and sources.



- 1 Query and result count.
- 2 Matching stories use the same card as the home page.

9. Inbox

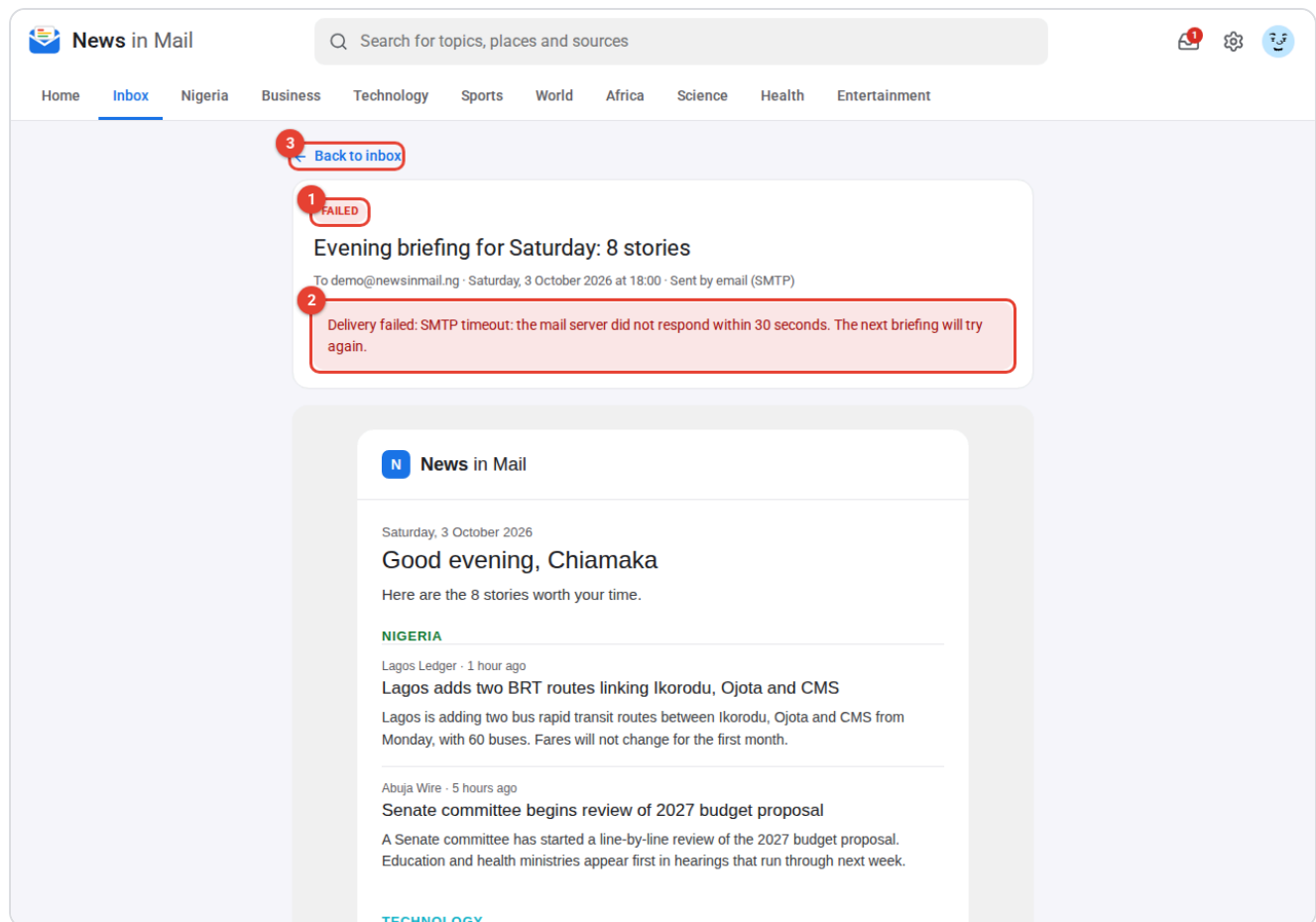
Every email the app has produced for this user, including problems.



- 1 Counts of sent, failed and skipped briefings.
- 2 A failed delivery shows the SMTP error.
- 3 A skipped slot: no new stories, so nothing was sent.
- 4 Newest first. Morning, evening and test emails have different icons.

10. A failed briefing

Opening a failed briefing shows what went wrong. The scheduler moves on to the next slot.



- 1 Status chip.
- 2 The delivery error from the mail server.
- 3 Back to the inbox.

11. Settings

The same preferences form as onboarding, plus pause and resume.

News in Mail Search for topics, places and sources

Home Inbox Nigeria Business Technology Sports World Africa Science Health Entertainment

Settings

Change your topics and when your briefings arrive.

Topics
Pick the subjects you want in your briefings.

1

World Nigeria ✓ Africa
Business ✓ Technology ✓ Science
Health Sports ✓ Entertainment

Delivery times
We send at most two emails a day.

2

☒ **Morning briefing**
What happened overnight 7:00 AM

☒ **Evening briefing**
Catch up on the day 6:00 PM

Time zone
Africa/Lagos

3 **Save changes**

Next email
6:00 PM
Evening briefing, today, to **demo@newsinmail.ng**
[Send a test email now](#)

Pause briefings
Going on holiday? Pause all emails and resume any time.
4 **Pause briefings**

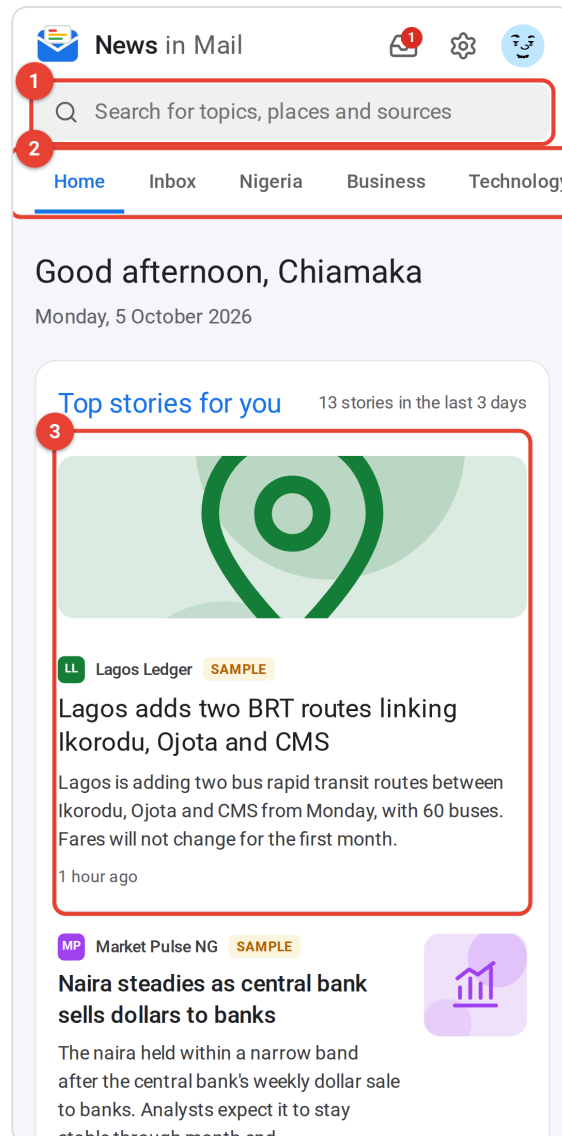
- 1 Change topics at any time.
- 2 Change or switch off a briefing.
- 3 Saves and shows a confirmation.
- 4 Pause all emails, for example on holiday.

5. Mobile view

Captured on a Pixel 7 sized screen (412 by 915). Every grid uses `minmax(0, 1fr)` columns so long headlines wrap instead of pushing the page sideways. A Playwright test checks that no page scrolls sideways.

1. Mobile: home

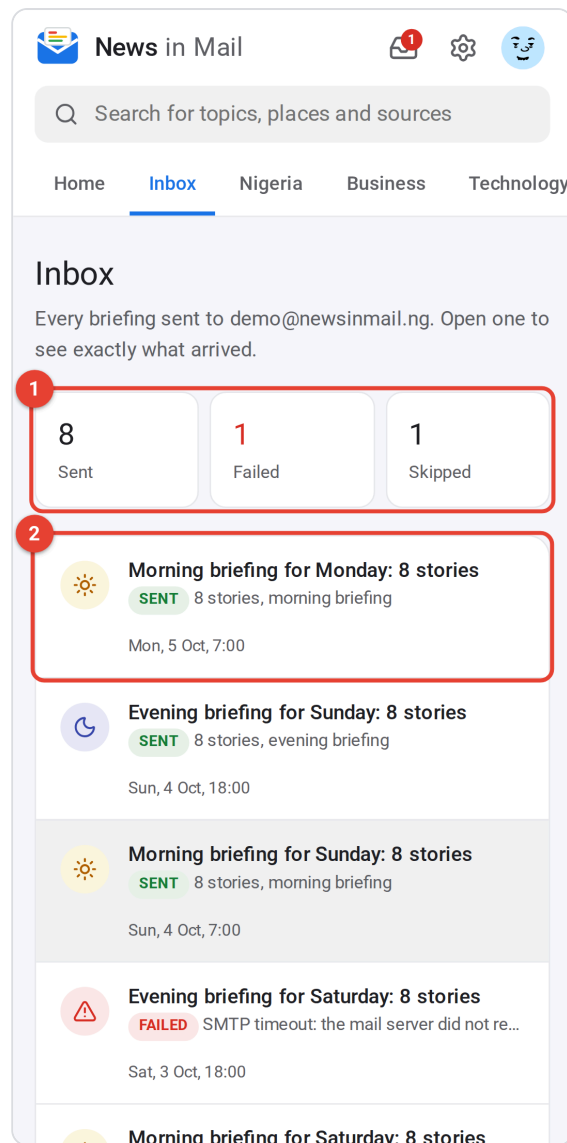
On a phone the search bar drops below the logo and the tabs scroll sideways.



- 1 Full-width search.
- 2 Swipeable section tabs.
- 3 Stories stack in one column.

2. Mobile: inbox

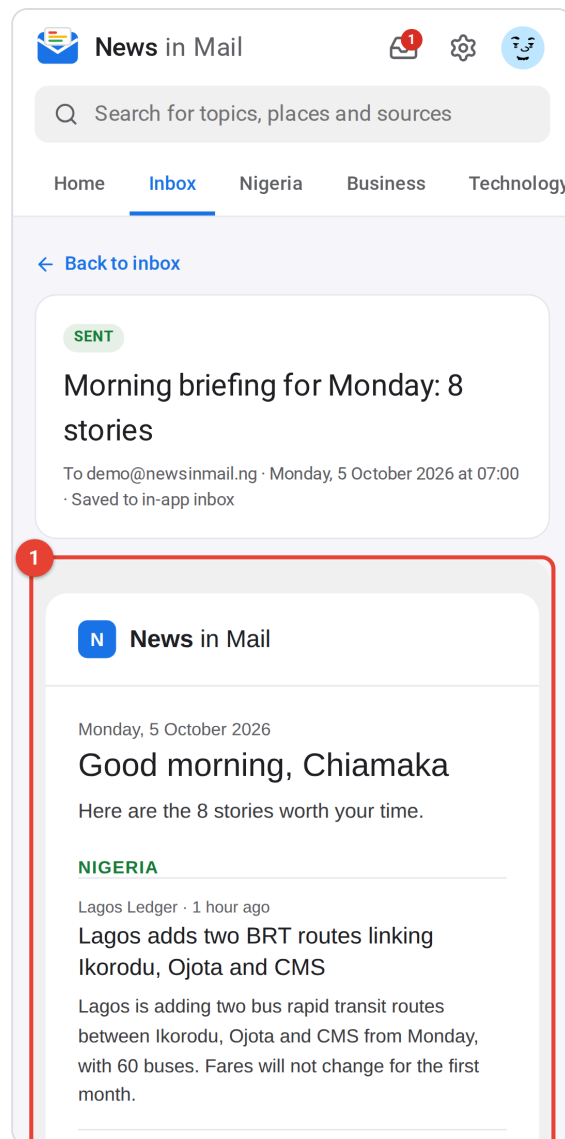
The inbox keeps the status chip and time under each subject.



- 1 Counts stay in one row.
- 2 Rows wrap instead of overflowing.

3. Mobile: an email

Emails are built with tables and inline styles so they also read well in phone mail apps.



1 The email at phone width.

6. Running locally

```
# 1. Database
service postgresql start
sudo -u postgres psql -c "CREATE DATABASE newsinmail;" -c "CREATE DATABASE newsinmail_test;"

# 2. App
cp .env.example .env          # set DATABASE_URL, SESSION_SECRET, OPENROUTER_API_KEY
npm install
npm run db:migrate
npm run db:seed               # demo users, sample stories, email history
npm run ingest                # optional: pull live stories now
npm run dev                   # http://localhost:3000
```

Variable	Meaning
DATABASE_URL	Postgres connection string
SESSION_SECRET	Secret used to sign session cookies (16+ characters)
OPENROUTER_API_KEY	Enables AI summaries. Without it, the first sentences of each story are used
OPENROUTER_MODEL	Optional. Default anthropic/claude-sonnet-5.5
SMTP_URL, MAIL_FROM	Optional. Real email delivery. Without them emails go to the in-app inbox
NEWSINMAIL_TODAY / NEWSINMAIL_NOW	Pin the date or exact time for demos and tests
NEWSINMAIL_SCHEDULER	"on" to run the scheduler in development, "off" to disable it
CRON_SECRET	Enables POST /api/cron to run the scheduler on demand

7. Testing

Suite	Tool	Tests	What it covers
Unit	Vitest	52	Schedule rules, story selection, text cleaning, email rendering, validation, session tokens, clock override, AI reply parsing
Integration	Vitest + Postgres	9	Seed data, sending a digest, skipping, no repeated stories, scheduler idempotency, paused users, user scoping, duplicate protection, RSS parsing
End to end	Playwright (desktop)	14	Landing, demo login, wrong password, inline sign-up errors, full onboarding with test email, home, inbox, failed email, test email button, settings and pause, tabs and search, 404 for other users' emails
End to end	Playwright (mobile, Pixel 7)	2	No sideways scrolling on four pages, opening an email from the inbox

Total: 77 tests, all passing.

```
npm test          # unit + integration (uses newsinmail_test)
npm run build
npm run test:e2e  # starts the app on port 3100 against newsinmail_e2e
```

Tests run offline (`NEWSINMAIL_OFFLINE=1`) with a fixed clock, so results do not depend on the news of the day.

8. Deployment

The app runs on Railway in the "school-projects" project as the service **news-in-mail**. Railway deploys automatically from the `main` branch of the GitHub repository.

- **Build:** `npm run build`
- **Start:** `npm run start:railway`: run migrations, seed demo data only if the database has no users, then `next start -H 0.0.0.0`.
- **Health check:** `/api/health` runs `select 1` against the database and returns `{"status":"ok"}`.
- **Variables:** `DATABASE_URL` (built from the Postgres service's `{{Postgres.*}}` references), `SESSION_SECRET` (random), `NODE_ENV=production`, `OPENROUTER_API_KEY`, `CRON_SECRET`.

The project had reached Railway's limit of 10 volumes, so a new Postgres service could not be created. The app uses the existing **Postgres** service but in its own database called `newsinmail`, which the migrate script creates on first start. Other apps' data is not touched.

Public URL: <https://news-in-mail-production.up.railway.app>

9. Five-minute presentation script

Time	What to do and say
0:00 - 0:30	Open the landing page. "News in Mail sends you a short news briefing every morning and evening, only on the topics you pick, summarized by AI from free news sources."
0:30 - 1:30	Click Create a free account, sign up with your own email. On onboarding pick two or three topics, show morning and evening times and the time zone. Click Finish. "A test email is sent straight away." Show the email: greeting, AI intro, stories grouped by topic, links to the source.
1:30 - 2:15	Sign out, sign in as the demo user (already filled in). Show the Home page: Google News style tabs, top stories with summaries, Next email card, topics and recent emails. Click a topic tab and do a search for "Lagos".
2:15 - 3:00	Open the Inbox. "Every briefing is recorded." Point to the sent, failed and skipped counts. Open the failed one: the SMTP error is shown. Open a skipped one: no new stories, so nothing was sent.
3:00 - 3:45	Explain the logic with the diagram in section 3: feeds are read every 20 minutes, the scheduler checks each user's local time every 5 minutes, picks fresh stories round-robin across topics, never repeats a story, summarizes once with Claude through OpenRouter, and a unique index stops duplicate emails.
3:45 - 4:30	Open Settings, change the evening time and save; the Next email card updates. Pause and resume. Show the site on a phone (or browser device mode).
4:30 - 5:00	Close with quality: 77 automated tests (unit, integration on real Postgres, Playwright on desktop and mobile), deployed on Railway with a database health check. Take questions.

10. Decisions and limits

- **Email delivery:** no mail account was provided, so emails are stored in an in-app inbox that shows them exactly as a mail app would. Setting `SMTP_URL` (for example a Gmail app password or a Brevo or Resend SMTP URL) sends real email with no code change.
- **Default times:** 7:00 AM and 6:00 PM, Africa/Lagos time zone. Morning must be before noon, evening at noon or later.
- **AI model:** Claude Sonnet 5.5 through OpenRouter, Gemini Flash as fallback. Summaries are made only for stories that go into an email and are cached, which keeps cost low.
- **Images:** news photos are not loaded (hotlinking is unreliable and blocked in the build sandbox). Each topic has its own SVG illustration instead.
- **Sample stories:** seeded stories use made-up sources and are labeled "Sample" so they are not mistaken for real news. Live stories replace them at the top as soon as the feeds are read.
- **Scheduler:** runs inside the web server (Next.js instrumentation) every 5 minutes, which suits a single Railway instance. The unique index makes it safe if more instances are added.